FULL STACK WEB DEVELOPMENT

STUDENT MANAGEMENT SYSTEM

A Full-Stack Web Application using React, Node.js, Express.js and MongoDB Atlas

Submitted by: Bhuvan Teja Jakkamsetti

Roll Number: A24126552084

Class / Section: CSM-B

Department: CSM

College: AnilNeerukonda Institute of SCiences and Technology

Faculty: Santosh

Academic Year: 3/4

PART A. Comprehensive Question

Case Study: Student Management System

Educational institutions maintain student information such as roll number, name, branch, year of study, marks and email address. Maintaining these records manually can make it difficult to add new entries, locate existing records, correct information and keep data consistent.

Design and develop a web-based Student Management System using React, Node.js, Express.js and MongoDB. The application must provide a user interface for managing student records, expose REST API endpoints for CRUD operations, store records in a MongoDB database, and be deployed using cloud platforms.

The application requirements are:

- Store student details including roll number, name, branch, year, marks and email.
- Create and use a MongoDB database and students collection with an appropriate document structure.
- Perform CRUD operations: add, display, update and delete student records.
- Provide API routes to retrieve all students and an individual student by ID.
- Connect the React frontend to the Express backend using HTTP requests.
- Deploy the frontend and backend separately and connect the backend to MongoDB Atlas.
- Test the deployed application and verify that records persist in the database.

PART B. Worked Solution

1. Analyse the requirements

The proposed Student Management System is a web-based application designed to maintain student academic and contact details in a central database. It replaces manual record handling with a browser-based interface through which a user can add, view, update and delete student records.

Functional requirements

- Add a student record with roll number, name, branch, year, marks and email.
- Display all stored student records in a table.
- Retrieve an individual student record through its MongoDB document identifier.
- Update an existing student record.
- Delete a student record after user confirmation in the frontend.
- Persist records in MongoDB Atlas and retrieve them through the Express API.

Technology requirements

- React and Vite for the frontend user interface.
- Node.js and Express.js for the backend REST API.
- Mongoose for schema definition and MongoDB communication.
- MongoDB Atlas for cloud database storage.
- Render for backend hosting and Vercel for frontend hosting.

Note: The application is a student-record management system. It does not implement authentication, role-based access, analytics dashboards, CSV export or aggregation reports; these are outside the verified implementation supplied.

2. Design the MongoDB document

Each student is stored as a document in the students collection within the studentDB database. Mongoose defines the structure and validation rules through the Student schema.

```
{
  "rollNo": "A24126552084",
  "name": "Example Student",
  "branch": "CSE-AIML",
  "year": 3,
  "marks": 88,
  "email": "student@example.com"
}
```

Field	Data type	Rule / purpose
rollNo	String	Required; unique
name	String	Required
branch	String	Required
year	Number	Required
marks	Number	Required
email	String	Required
_id	ObjectId	Generated by MongoDB/Mongoose

The _id value generated for each document is used by the application when requesting an individual record, updating a record or deleting it. The rollNo field is unique, preventing duplicate roll numbers when the database unique index is active.

3. Create the database and collection

A MongoDB Atlas cluster was created for cloud database hosting. A database user and network access rule were configured so the deployed backend can connect to the cluster. The application connects through the MONGODB_URI environment variable; the credential itself is intentionally not included in this report.

```
Database: studentDB
Collection: students
```

Mongoose connects to the Atlas cluster using mongoose.connect(process.env.MONGODB_URI). When the connection succeeds, the Express server begins listening on the configured port. If the database connection fails, the backend logs the error and exits rather than starting an API that cannot access its database.

Note: The application uses Mongoose to create and use the collection through the Student model. No hard-coded database password or live connection string should be placed in source code or screenshots.

4. Perform student record CRUD operations

CRUD represents Create, Read, Update and Delete. These operations are exposed through the Express routes mounted at /api/students.

Operation	HTTP method	Endpoint	Purpose
Create	POST	/api/students	Create a student document
Read all	GET	/api/students	Return all student documents
Read one	GET	/api/students/:id	Find one document by its _id
Update	PUT	/api/students/:id	Update document fields
Delete	DELETE	/api/students/:id	Remove a document

Implementation behaviour

- Create uses Student.create(req.body) and returns HTTP 201 when successful.
- Read all uses Student.find() and returns the student array.
- Read one uses Student.findById() and returns 404 when no matching student exists.
- Update uses findByIdAndUpdate() with { new: true, runValidators: true }.
- Delete uses findByIdAndDelete() and returns a success message when deletion completes.
- Route handlers catch errors and return JSON error messages with appropriate status codes.

The frontend calls these endpoints through fetch() functions in studentService.js. The React component refreshes the student list after successful create, update and delete operations.

5. Student schema constraints and data integrity

The Student schema defines six required business fields. rollNo is marked unique, while the other fields are required. year and marks are stored as numbers; the frontend converts their form values to Number before sending the request.

```
const studentSchema = new mongoose.Schema({
  rollNo: { type: String, required: true, unique: true },
  name:   { type: String, required: true },
  branch: { type: String, required: true },
  year:   { type: Number, required: true },
  marks:  { type: Number, required: true },
  email:  { type: String, required: true }
});
```

Note: The supplied implementation does not define custom email-format validation, mark-range validation in the Mongoose schema, timestamps, or additional secondary indexes. The browser form includes required fields, email input type, and numeric constraints for year and marks.

6. Develop the Node.js and Express.js backend

The backend is maintained in the backend folder. The project uses Express.js to create the HTTP server, cors to enable cross-origin requests, mongoose to communicate with MongoDB, and dotenv to load local environment variables.

Backend initialization

```
npm init -y
npm install express mongoose cors dotenv
npm install -D nodemon
```

Server configuration

```
const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");
require("dotenv").config();

const studentRoutes = require("./routes/studentRoutes");
const app = express();

app.use(cors());
app.use(express.json());
app.use("/api/students", studentRoutes);

const PORT = process.env.PORT || 5000;
```

The server connects to MongoDB Atlas before calling app.listen(). The root route (/) returns the message “Student Management System Backend is running”. The student routes are grouped under /api/students for a consistent REST API structure.

Backend project structure

```
backend/
  models/
    student.js
  routes/
    studentRoutes.js
  server.js
  package.json
  .env  (local/private; not committed)
```

7. Develop the React frontend

The frontend is a Vite-powered React application. App.jsx maintains the student list, form values, edit state, status message and loading state using useState. useEffect loads student records when the page is first rendered.

Frontend capabilities

- Displays the Student Management System heading and student-record form.
- Collects roll number, student name, branch, year, marks and email.
- Displays stored records in a table with Edit and Delete actions.
- Uses the same form for both adding and updating records.
- Displays operation messages and a loading state.
- Asks for confirmation before deleting a student.

API service configuration

```
const API_URL =
  import.meta.env.VITE_API_URL ||
  "http://localhost:5000/api/students";
```

The VITE_API_URL environment variable allows the frontend to use the deployed Render API in production while retaining the localhost API fallback for local development. The service module contains separate functions for getStudents, addStudent, updateStudent and deleteStudent.

Frontend project structure

```
frontend/
  src/
    services/
      studentService.js
    App.jsx
    App.css
    main.jsx
  package.json
  vite.config.js
```

8. Run and test the application locally

For local development, the backend and frontend are started in separate terminals. The backend reads MONGODB_URI from backend/.env and connects to MongoDB Atlas. The frontend runs through the Vite development server and sends requests to the local Express API.

Backend commands

```
cd backend
npm install
npm run dev
```

Frontend commands

```
cd frontend
npm install
npm run dev
```

Local verification procedure

1. Open the Vite development URL displayed in the terminal.
2. Add a student with all required fields and verify that the record appears in the table.
3. Refresh the page and verify that the record is loaded again from the database.
4. Edit the record and verify the updated values.
5. Delete the record and verify that it is removed from the list and database.
6. Check the GET /api/students endpoint to inspect the returned JSON.

Note: The report records the local testing workflow. The successful end-to-end testing outcome was confirmed by the project owner during the project walkthrough; screenshots should be inserted from the user's own browser and Atlas session.

9. Deploy the application using cloud platforms

The completed application uses a three-part cloud architecture. Vercel hosts the React frontend, Render hosts the Node.js/Express backend, and MongoDB Atlas stores student documents. The frontend communicates with the backend using HTTP requests to the deployed API.

Deployment configuration

Platform	Component	Configuration / role
Vercel	React frontend	Root directory: frontend; Vite build; output: dist
Render	Express backend	Root directory: backend; build: npm install; start: npm start
MongoDB Atlas	Database	Cloud-hosted MongoDB cluster; connection via MONGODB_URI

Deployment sequence

7. Create the MongoDB Atlas cluster, database user and network access configuration.
8. Push the project source to the GitHub repository without committing .env files.

9. Create a Render Web Service connected to the repository; use backend as the root directory, npm install as the build command and npm start as the start command.
10. Add MONGODB_URI as a Render environment variable. Render supplies the runtime PORT value.
11. Deploy the backend and verify the /api/students endpoint.
12. Import the repository into Vercel and set frontend as the root directory with the Vite build configuration.
13. Set VITE_API_URL in Vercel to the deployed Render API endpoint ending in /api/students.
14. Deploy the frontend and test create, read, update and delete operations through the public website.

Verified deployment addresses

Frontend (Vercel): https://student-management-system-one-lilac.vercel.app/

Backend API (Render): https://student-management-system-8w6u.onrender.com/api/students

Note: The project source uses app.use(cors()) without an origin allow-list. For a production system, CORS should be restricted to the trusted frontend origin. The Atlas network access rule should also be narrowed where practical, and database credentials must remain private.

10. Local application execution versus cloud deployment (Faculty Question Q2)

Local application execution means running the frontend and backend on the developer's own computer for development and testing. In this project, the Vite frontend runs locally and sends requests to the Express server at localhost:5000. The backend connects to MongoDB Atlas using a private environment variable.

Cloud deployment makes the application components available through internet-accessible hosting services. In the completed project, Vercel serves the React frontend, Render runs the Node.js/Express backend, and MongoDB Atlas provides the cloud database.

Aspect	Local execution	Cloud deployment in this project
Frontend address	Local Vite development URL	Public Vercel URL
Backend address	http://localhost:5000	Public Render service URL
Availability	Available while local development server and computer are running	Accessible over the internet, subject to hosting service availability
Purpose	Coding, debugging and local testing	Sharing and using the application remotely
Configuration	Local .env and Vite settings	Render environment variables and Vercel VITE_API_URL
Database	Can connect to Atlas using local credentials	Render backend connects to Atlas using MONGODB_URI

Purpose of Vercel for the frontend

Vercel hosts and serves the built React/Vite frontend. It provides a public website URL so users can open the Student Management System through a browser without running the frontend development server on their own computers. The VITE_API_URL setting directs frontend requests to the deployed backend.

Purpose of Render for the backend

Render runs the Node.js and Express.js server as a web service. It provides a public API endpoint that receives requests from the React frontend, executes the student CRUD route handlers, communicates with MongoDB Atlas through Mongoose, and returns JSON responses.

How the deployed components work together

15. A user opens the public Vercel frontend in a browser.
16. React sends a request to the Render API endpoint.
17. Express receives the request at /api/students and dispatches it to the appropriate route.
18. The route uses the Mongoose Student model to read or modify data in MongoDB Atlas.
19. The backend returns a JSON response to the frontend.
20. React updates the interface to display the latest student records.

Therefore, Vercel and Render serve different but complementary purposes: Vercel delivers the user interface, while Render executes the backend application logic and exposes the API. MongoDB Atlas persists the records independently of either hosting service.

11. Expected final outcome

The completed Student Management System provides a browser-based interface for maintaining student records. The React frontend communicates with the Express backend, and the backend uses Mongoose to persist data in MongoDB Atlas.

- A public React frontend hosted on Vercel.
- A public Express API hosted on Render.
- A MongoDB Atlas database containing the student collection.
- Create, read-all, read-one, update and delete API operations.
- A responsive student form and records table with edit and delete actions.
- A deployment architecture that separates frontend hosting, backend execution and database storage.

The project demonstrates the integration of frontend development, REST API design, database modelling, environment-based configuration, GitHub source control and cloud deployment.

PART C. Output Screenshots (Test Case Results)

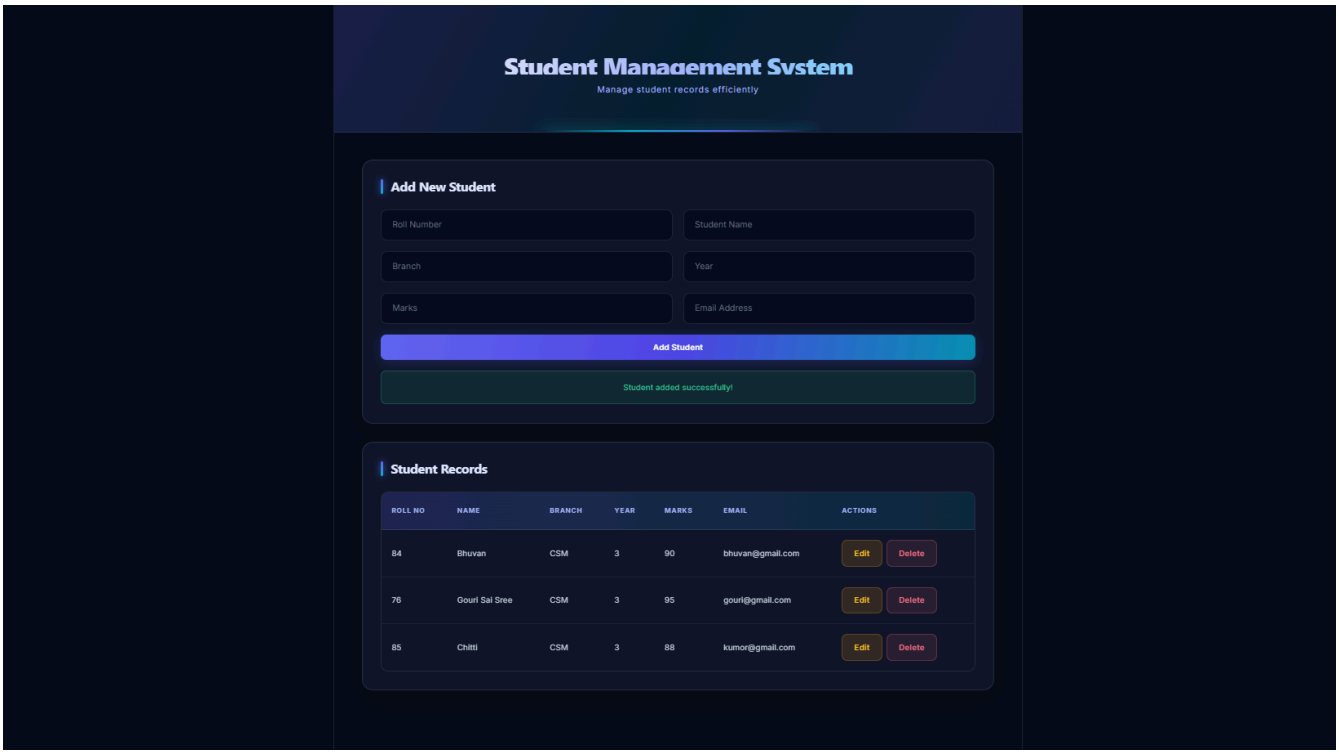


Figure: Student Management System Dashboard. Shows the deployed React application with the student form and student records table.

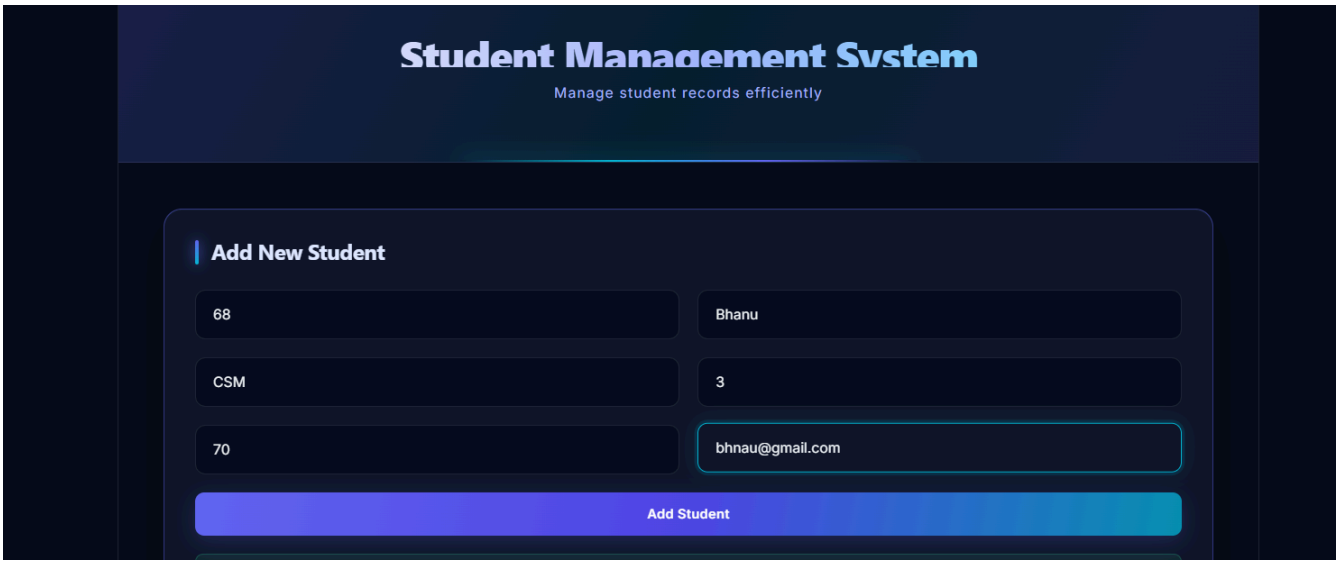


Figure: Add Student Operation. Shows the form populated with a test student record and the successful creation result.



Figure: Student Records Display. Shows the student record appearing in the table after it has been retrieved from the backend.

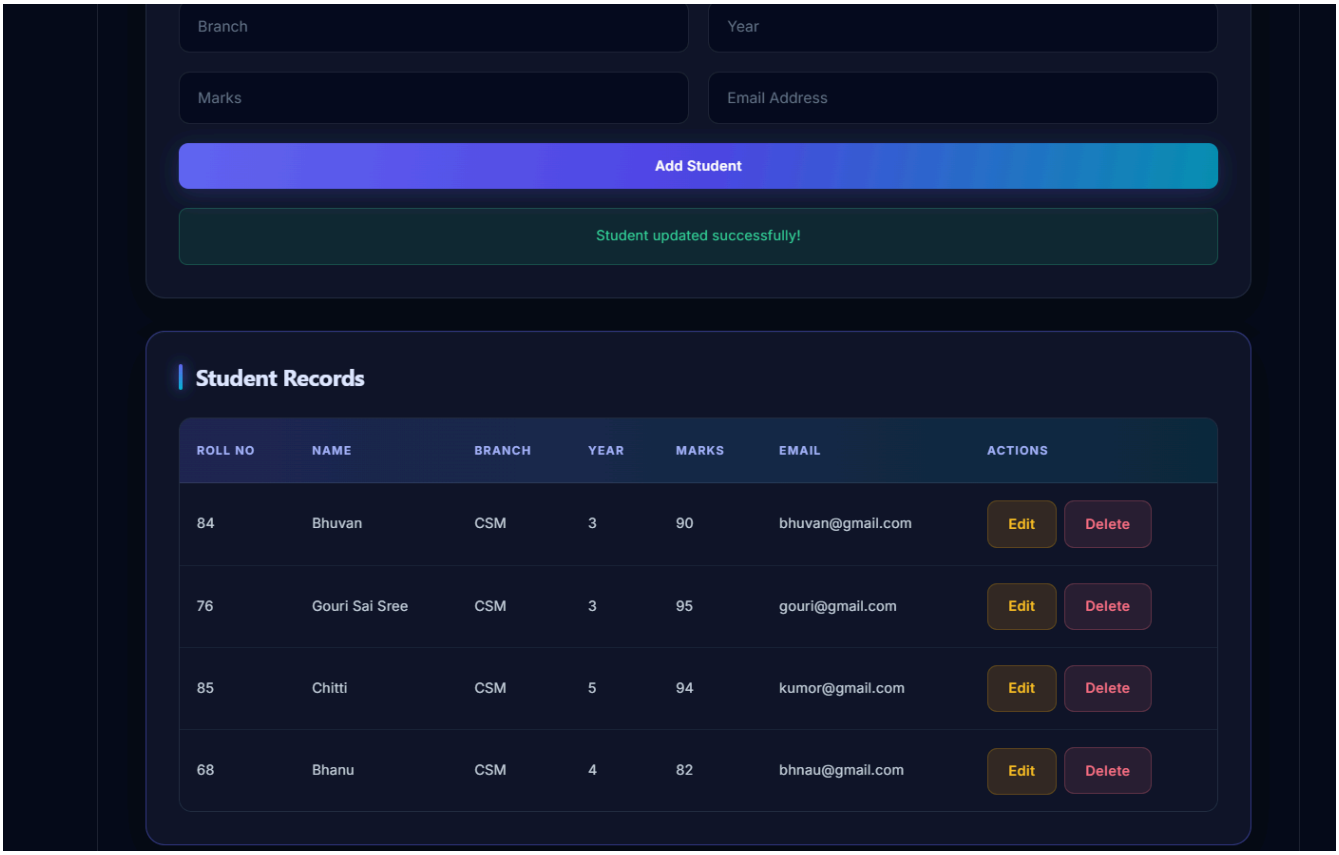


Figure: Update Student Operation. Shows an existing student record being edited and the updated values displayed.

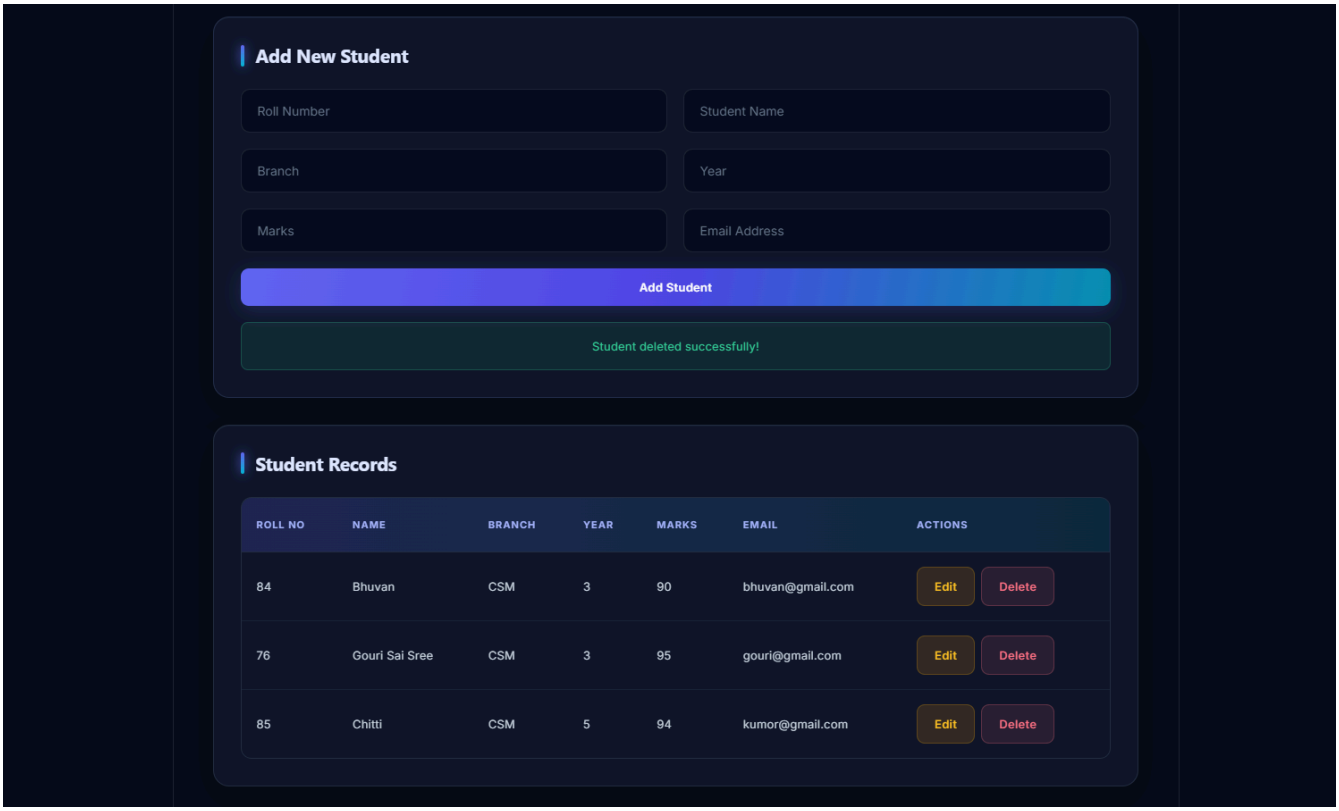


Figure: Delete Student Operation. Shows the record removed from the student table after the delete operation.

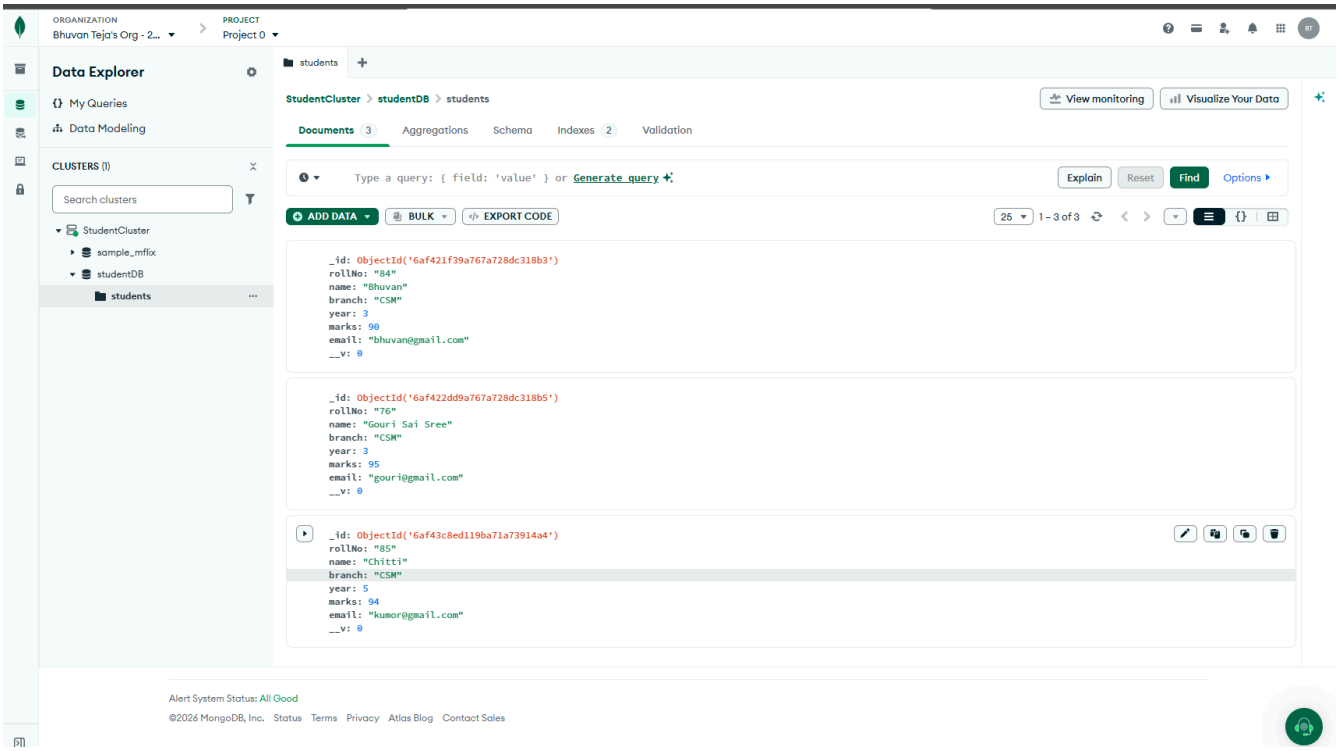


Figure: MongoDB Atlas – students Collection. Shows the studentDB database and students collection containing the persisted student document. Hide any credentials or sensitive connection details.

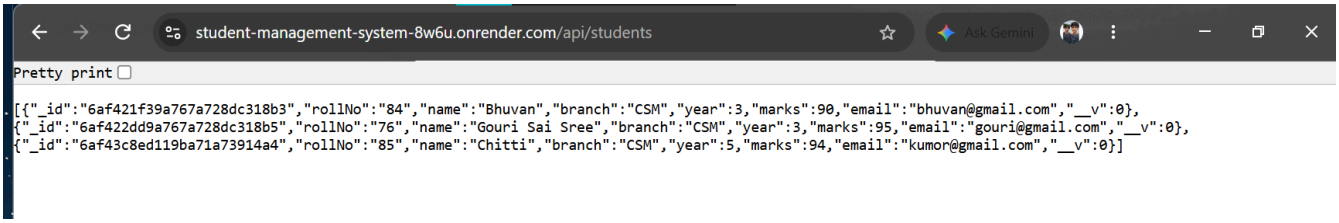


Figure: Backend API Response on Render. Shows the deployed /api/students endpoint returning JSON student records.

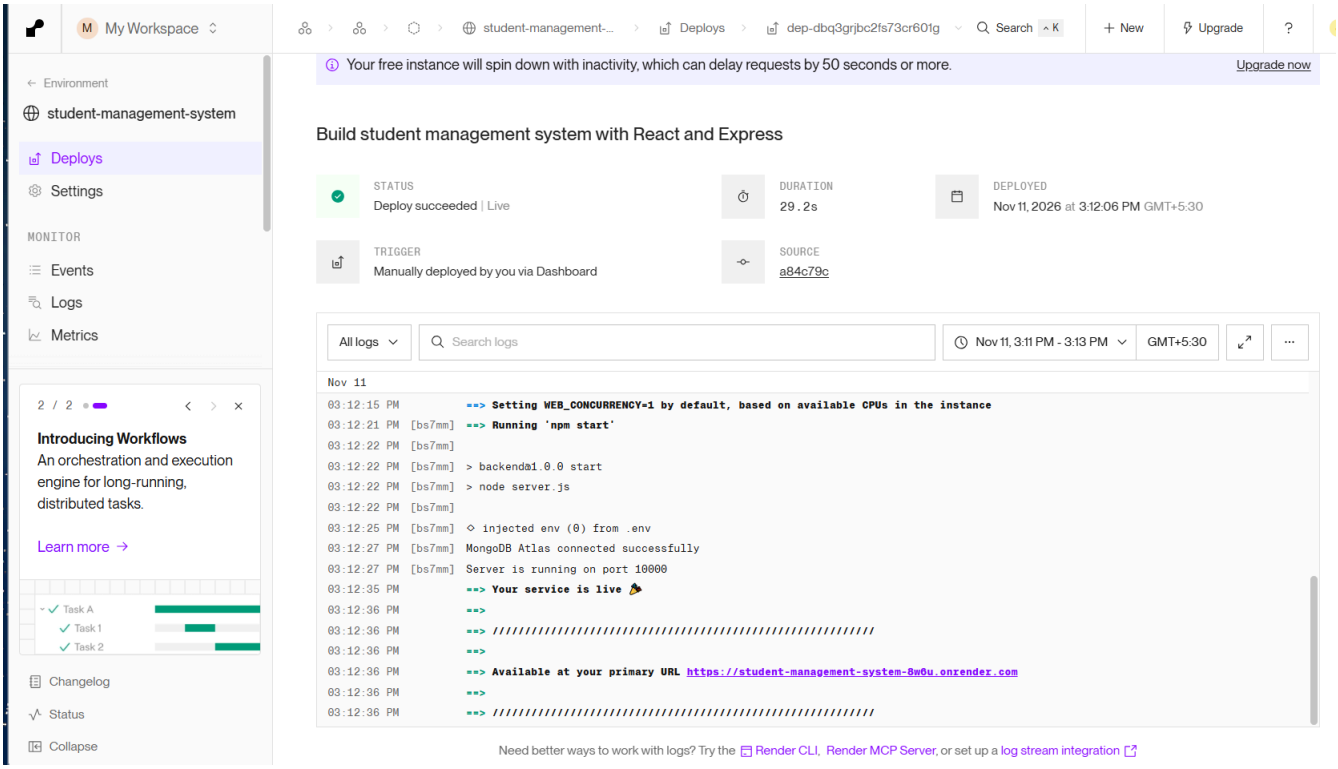


Figure: Render Backend Deployment Status. Shows the Render service dashboard with the backend deployment marked Live.

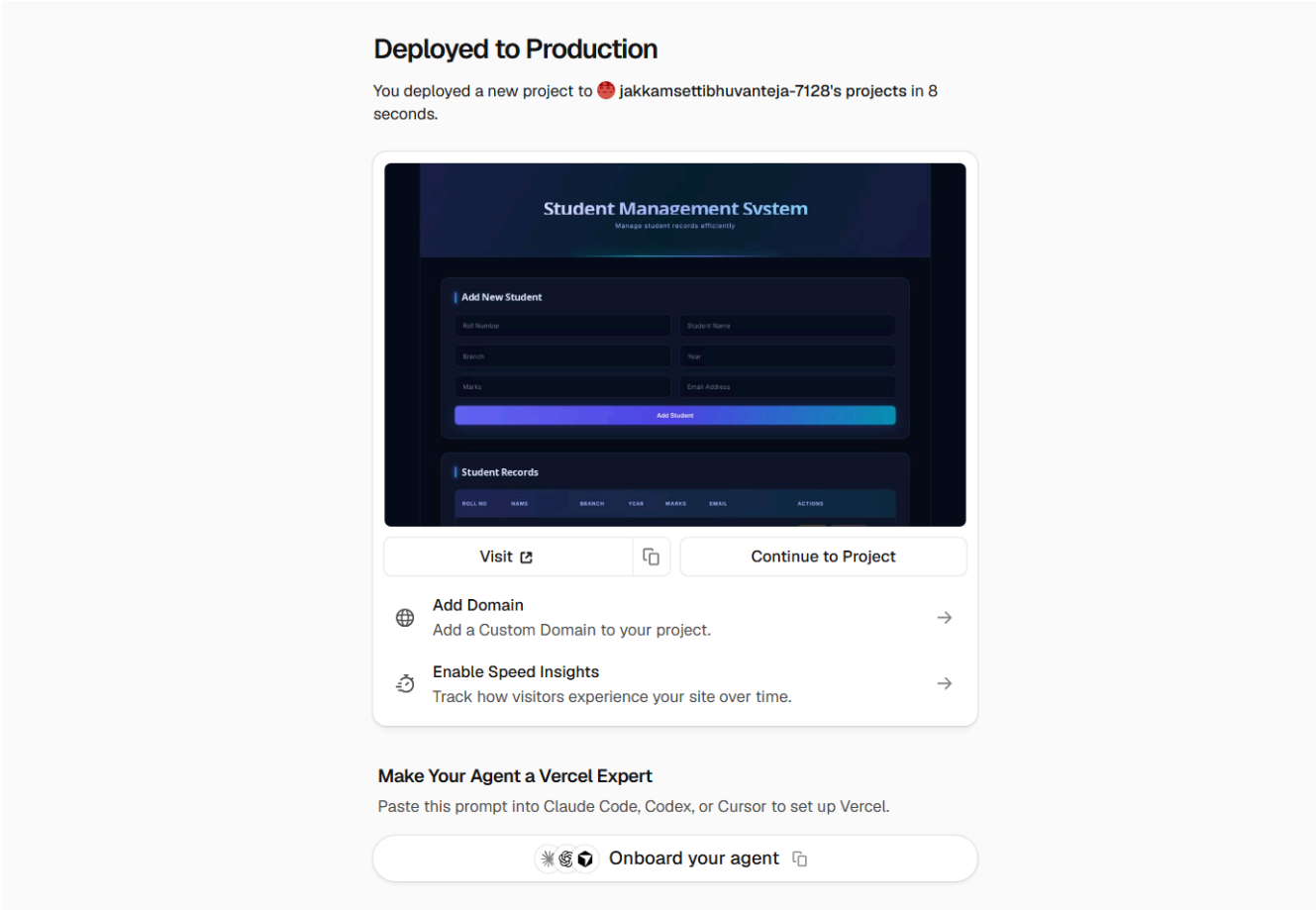


Figure: Vercel Frontend Deployment Status. Shows the Vercel project deployment marked Ready/Successful and the deployed project domain.

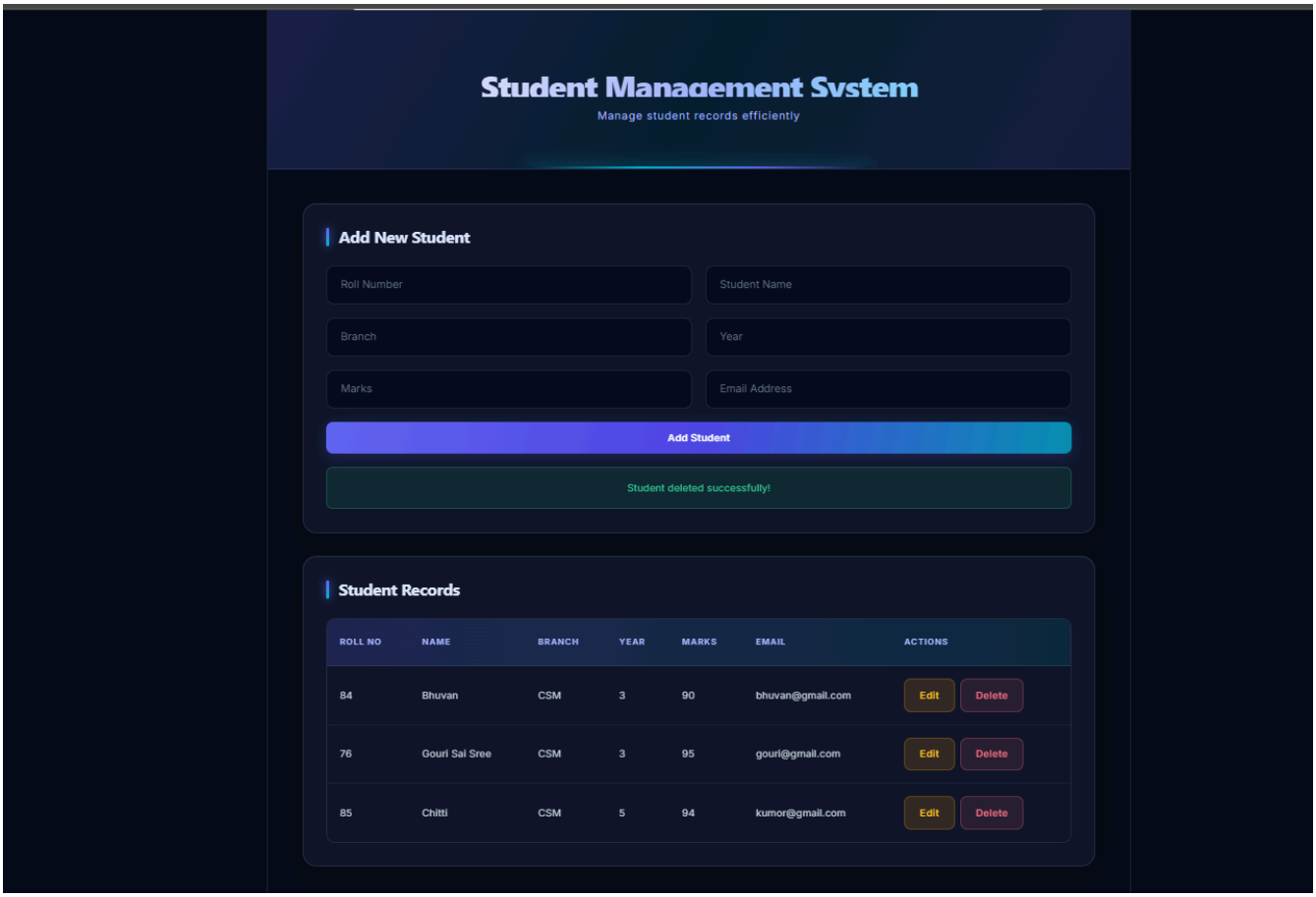


Figure: Deployed Application End-to-End Test. Shows the live Vercel application after successful interaction with the Render API and Atlas database.

References and Project Links

- GitHub Repository: https://github.com/jakkamsettibhuvanteja-ui/student-management-system
- Frontend (Vercel): https://student-management-system-one-lilac.vercel.app/
- Backend API (Render): https://student-management-system-8w6u.onrender.com/api/students
- MongoDB Documentation: https://www.mongodb.com/docs/
- React Documentation: https://react.dev/
- Express Documentation: https://expressjs.com/
- Vercel Documentation: https://vercel.com/docs
- Render Documentation: https://render.com/docs